

1. INHALTSVERZEICHNIS & PRÜFUNGS-KOMPASS

BASICS & STRINGS: Typen (2), Operatoren (3), LEGB (4), Math (5), Strings (7), f-Strings (8), Escape (9), Regex (10), Cleaning (11), Slicing (12), Match/Case (13), Loops (14), I/O (15)

S. 1

KOLLEKTIONEN & ITERATION: Listen in-place (17), Sort/Keys (18), Tupel (19), Copy (20), Chunking (21B), Dicts & Merge (22, 23), Sets (24), Comprehensions (30), groupby (31B)

S. 2

FUNKTIONEN, OOP & FEHLER: *args/**kwargs (32), Mutable Default (33), First-Class (34), Dekoratoren (35), Klassen (37), @property (38), super() (42), Dunder (43), try/except (44)

S. 3

DATEIEN, PAKETE & KLAUSUR 2025: Datei-I/O (46), CSV von Hand (48), Pakete (49), ★ **A1 Syntax (51)**, ★ **A2 CSV Spaltentausch (52)**, ★ **A3 OOP (53)**, ★ **A4 Paket (54)**, Notfall-Joker (55), H:\ (56)

S. 4

Regeln: 2 Blätter A4 beidseitig erlaubt | **STRENG VERBOTEN:** NumPy & Pandas (= 0 Pkt!) | Alles nach H: sichern!

2. DATENTYPEN, IDENTITÄT & SPEICHER

• **Dynamisch & streng getypt:** Typ wird zur Laufzeit ermittelt; kein implizites Casting ('3'+1 wirft TypeError).

• **Basistypen:** int (beliebig groß), float (64-Bit), complex (1+2j), bool, str, NoneType.

• **Falsy-Werte:** 0, 0.0, 0j, '', [], (), {}, set(), None (alles andere ist True).

• **Gleichheit (==) vs. Identität (is):**

```
a = [1, 2]; b = [1, 2]
a == b # True (gleicher Wert)
a is b # False (verschiedene Adressen)
id(a) == id(b) # False (is vergleicht id)
# Caching: Ints -5..256 & kurze Strings:
x = 10; y = 10; x is y # True!
# Typ-Prüfung: type() vs. isinstance():
type(True) is int # False (strikter Typ)
instance(True, int) # True (bool erbt von int!)

• Typ-Casting: int('42'), float('3.14'), str(100), bool(1).


```

3. OPERATOREN & AUSDRÜCKE

• **Arithmetik:** +, -, *, / (immer float!), // (Ganzzahldivision), % (Modulo), ** (Potenz).

• **Bit-Operatoren:** & (AND), | (OR), ^ (XOR), ~ (NOT), <<, >>.

• **Logik:** and, or, not (Kurzschluss: bricht ab sobald Ergebnis feststeht).

• **Vergleiche:** ==, !=, <, <=, >, >= | Chained: 1 < x <= 10.

```
7 / 2 # 3.5 (echte Division)
7 // 2 # 3 (Ganzzahlig)
7 % 2 # 1 (Rest)
2 ** 8 # 256 (Potenzierung)
x = (y := 5) * 2 # Walrus: y=5, x=10
```

4. NAMENSRÄUME & LEGB-REGEL

Auflösung: Local → Enclosing → Global → Built-in.

• global x: globale Modulvariable in Funktion modifizieren.

• nonlocal x: umschließende Variable in Nested Function modifizieren.

```
x = 5
def outer():
    x = 10
    def inner():
        nonlocal x; x = 20
    inner(); return x
outer() # 20 (Modul-x bleibt unberührt bei 5)
# Introspektion: locals(), globals(), dir(obj), vars(obj)
```

5. MATHE-BUILTINS & MATH-MODUL

• **Builtins:** abs(-5), round(3.1415, 2), divmod(17, 5) → (3, 2), pow(2, 3, 5) → 3, sum([1,2,3]), min(s), max(s).

• **Modul math:** (Standardbibliothek):

```
import math
math.hypot(3, 4) # 5.0 (Euklidische Distanz: sqrt(x^2+y^2))
math.sqrt(16) # 4.0 (Quadratwurzel)
math.pi, math.e # 3.14159..., 2.71828...
math.ceil(2.3) # 3 (Aufrunden) | floor(2.9) # 2
math.radians(180) # 3.14159... | degrees(pi) # 180.0
math.gcd(24, 36) # 12 (Größter gemeinsamer Teiler)
math.isclose(0.1+0.2, 0.3) # True! (Float-Vergleich!)
```

6. VARIABLEN, SPEICHER & ZAHLENSYSTEME

• **Zuweisungen:** a, b = 1, 2 (Unpacking), a = b = 0 (geteiltes Objekt!).

• **Augmented:** x += 1 (bei int neu, bei Listen in-place wie extend!).

• **Zahlensysteme:**

```
bin(42) # '0b101010' (Binär)
hex(255) # '0xff' (Hexadezimal)
oct(64) # '0o100' (Oktal)
int('1010', 2) # 10 (Basis 2 zu int)
int('ff', 16) # 255 (Basis 16 zu int)
# Bit-Operationen: 5 & 3 -> 1, 5 | 2 -> 7, 1 << 3 -> 8
```

7. STRINGS IM DETAIL (METHODEN)

• **Unveränderlich (Immutable):** s[0]='a' → **TypeError!**

• **Suchen:** s.find('x') (-1 wenn fehlt!), s.index('x') (ValueError), s.count('x').

• **Trimmen:** s.strip(), s.lstrip(), s.rstrip() (Whitespace/Newlines entfernen).

• **Teilen:** s.split(','), s.rsplit(',', 1) (von rechts, max 1x).

• **Fügen:** ', '.join(parts) (nur über Strings!).

• **Zeilen:** lines = text.splitlines() (ohne Newlines).

• **Ersetzen:** s.replace('alt', 'neu') (neuer String!).

• **Prüfung:** .startswith(), .endswith(), .isdigit(), .isalpha(), .isalnum(), .isidentifier().

• **Case:** .lower(), .upper(), .title(), .capitalize(), .casefold() (für Vergleiche).

• **Ausrichten:** s.center(20, '-'), s.ljust(15), s.rjust(15), s.zfill(6).

```
s = 'Hallo Welt'
s.strip() # 'Hallo Welt'
'a,b,c'.split(',') # ['a', 'b', 'c']
'-'.join(['x','y']) # 'x-y'
'E-car'.casefold() == 'e-car' # True
'123'.zfill(5) # '00123'
```

8. STRING-FORMATIERUNG (F-STRINGS, .FORMAT)

• **f-Strings (Empfohlen ab Python 3.6):**

```
brand, kwh, n = 'Tesla', 16.148, 42
f'{brand}: {kwh:.2f} kWh' # 'Tesla: 16.15 kWh'
f'{n:05d}' # '00042' (5-stellig mit 0)
f'{kwh:>8.1f}' # rechtsbündig Breite 8
f'{brand!r}' # '"Tesla"' (mit Quotes)
f'{brand=}' # 'brand="Tesla"' (Debug)
# Expression direkt im f-String:
L = [10, 20, 30]
f'Summe: {sum(L)}, Min: {min(L)}' # 'Summe: 60, Min: 10'

• str.format():



```
'{} {}'.format('A', 'B') # 'A B'
'{0} {1} {0}'.format('X','Y')# 'X Y X'
'{b} {m}'.format(**{'b': 'Tesla', 'm': '3'})
'{:f}{w}'.format(5, f='0', w=5) # '00005'

• Traditionell mit %: '%s: %.2f' % ('Auto', 12.3).


```


```

9. ESCAPE-SEQUENZEN & ZEICHENKODIERUNG

• **\n:** Zeilenumbruch | **\t:** Tabulator | **\\:** Backslash | **\'**, **\"**: Quotes.

• **Raw-Strings (r'...'):** Backslashes bleiben wörtlich (ideal für Windows-Pfade & Regex!).

• **Tastatur-Kürzel:** **l:** AltGr + **B** (Mac: Opt+Shift+7) | **|:** AltGr + **<** (Mac: Opt+7)

```
path = r'c:\Users\Dokumente\data.txt' # kein Escaping nötig!
multi = 'Zeile 1\nZeile 2' # Mehrzeiler mit Newline
b = 'Text'.encode('utf-8') # bytes: b'Text'
s = b.decode('utf-8') # str: 'Text'
```

10. REGULÄRE AUSDRÜCKE (MODUL RE)

Musterbasierte Textsuche & Extraktion (Standard-Bibliothek):

```
import re
text = 'Messwert: X=12.5; Y=-4.2; Zeit=10:30'
# Suchen (erstes Vorkommen):
m = re.search(r'X=([0-9.]+)', text)
# Alle Vorkommen extrahieren:
nums = re.findall(r'[+]?d\d+\d+|\d+', text)
# ['12.5', '-4.2', '10', '30']
# Ersetzen & Teilen:
clean = re.sub(r'[;#]', '', text) # Ersetzen
tokens = re.split(r'[; +]', text) # Multi-Split
# Wichtige Symbole: \d Ziffer, \w Wort, ^ Start, $ Ende
```

11. STRING-PIPELINES & DATENBEREINIGUNG

Typischer Datenbereinigungsprozess für Klausurprüfungen:

```
raw_line = ' | 12.4 | -5.8 | NA ; 42.0 \n'
# 1. Whitespace & Newline trimmen:
clean = raw_line.strip()
# 2. Am Trennzeichen splitten:
tokens = [t.strip() for t in clean.split(';')]
# 3. Filtern & Konvertieren mit try/except:
values = []
for tok in tokens:
    try: values.append(float(tok))
    except ValueError: pass # ignoriert '', 'NA'
# values = [12.4, -5.8, 42.0]
result = f'Mittel: {sum(values)/len(values):.2f}'
```

11B. STRING-TRANSFORMATION & TRANSLATION

• **Zeichenersetzung mit Translation-Table:**

```
tbl = str.maketrans('aeiou', '12345') # Vokale zu Ziffern
'hallo welt'.translate(tbl) # 'h1ll4 w2lt'
# Umlaute säubern:
uml_map = str.maketrans({'ä': 'ae', 'ö': 'oe', 'ü': 'ue'})
'Müller'.translate(uml_map) # 'Mueller'
```

12. SEQUENZEN & SLICING

Gilt für Strings, Tupel und Listen:

• **Indizierung:** s[0] (erstes), s[-1] (letztes), s[-2] (vorletztes).

• **Slicing:** s[start:stop:step] (start inkl., stop exkl.).

```
s = [10, 20, 30, 40, 50, 60, 70, 80, 90]
s[:3] # [10, 20, 30] (erste 3)
s[-3:] # [70, 80, 90] (letzte 3)
s[2:6] # [30, 40, 50, 60] (von 2 bis vor 6)
s[::2] # [10, 30, 50, 70, 90] (jedes 2.)
s[::-1] # [90, 80, ..., 10] (Umkehren!)
# Slice-Zuweisung (nur bei Listen!):
s[2:5] = [0, 0, 0] # ersetzt Bereich
del s[2:5] # löscht Bereich
```

• **Operatoren:** x in s, x not in s, s + t, s * 3, len(s), min(s), max(s).

13. VERZWEIGUNGEN & FALLUNTERSCHIEDUNGEN

• **Mehrfachvergleiche in 1 Zeile:** 2 <= a <= 10.

• **Ternary-Operator:** x = a if bedingung else b.

• **Pattern Matching (ab Python 3.10):**

```
match point:
    case (0, 0): print('Ursprung')
    case (x, 0): print(f'Auf X-Achse: {x}')
    case (x, y) if x == y: print('Diagonale')
    case _: print('Sonstiger Punkt')
```

• **Short-Circuit:** a and b liefert erstes falsy oder letztes truthy Objekt.

14. SCHLEIFEN & KONTROLLFLUSS

• **range(start, stop, step):**

```
list(range(5)) # [0, 1, 2, 3, 4]
list(range(1, 11)) # [1, 2, ..., 10]
list(range(10, 0, -2))# [10, 8, 6, 4, 2]
```

• **break:** Schleife sofort beenden.

• **continue:** Rest des Durchlaufs überspringen.

• **for-else / while-else:** Läuft nur wenn kein break auftrat!

```
for x in [1, 3, 5]:
    if x % 2 == 0: break
else:
    print('Keine gerade Zahl gefunden!')
```

15. KONSOLEN-I/O & FORMATIERTE AUSGABE

• **Benutzereingabe:** s = input('Prompt: ') (liefert immer str; Typ-Cast: val = int(input())).

• **print()-Funktion:** Parameter sep, end, file, flush:

```
print('A', 'B', 'C', sep='; ') # 'A; B; C'
print('Lade...', end='', flush=True) # ohne Newline
import sys
print('Fehler!', file=sys.stderr) # Standard Error
# Tabellenartige Ausgabe mit format / f-string:
for name, val in [('Akku', 85.5), ('Motor', 120.0)]:
    print(f'{name:<10}: {val:>6.1f} kW')
```

16. SCHLEIFEN-MUSTER & AKKUMULATOREN

Effiziente Kontrollflussmuster ohne Drittmodule:

```
# Akkumulator (Summe / Zähler):
total = 0
for x in [10, 20, 30]: total += x
# Größtes Element manuell finden:
maximum = None
for val in [3, 9, 2, 7]:
    if maximum is None or val > maximum: maximum = val
# Mehrere Schleifen abbrechen (Flag / Return):
found = False
for i in range(5):
    for j in range(5):
        if i * j == 6: found = True; break
    if found: break
```

16B. OPERATOREN-RANGFOLGE (PRECEDENCE)

Reihenfolge der Auswertung (höchste zu niedrigste):

- () Klammern | 2. ** Potenzierung
- +x, -x, ~x (Unäre Operatoren) | 4. *, /, //, %
- +, - (Binär) | 6. <<, >> (Shifts)
- & | 8. ^ | 9. | (Bitweise)
10. ==, !=, <, <=, >, >=, is, in (Vergleiche)
11. not | 12. and | 13. or | 14. = Walrus

PYTHON 3 CHEAT-SHEET | BLATT 1 (SEITE 2/4): SEQUENZEN, KOLLEKTIONEN & ITERATION

TIPP: In-Place Methoden (sort, reverse, append) geben None zurück! sorted() & reversed() erzeugen neue Objekte.

17. LISTEN IM DETAIL (METHODEN)

In-Place vs. Functional:

• **In-Place Methoden verändern das Objekt selbst und geben None zurück!**

- L.append(x): hängt ein Element ans Ende an.
- L.extend(seq): hängt alle Elemente aus seq an (L += seq).
- L.insert(i, x): fügt x vor Index i ein.
- L.pop(=-1): entfernt & gibt Element zurück (Default: letztes).
- L.remove(x): entfernt erstes Vorkommen von x (ValueError wenn fehlt!).
- L.clear(): leert Liste (L[:] = []).
- L.index(x, [start]): Index von x.
- L.count(x): Häufigkeit von x.
- L.reverse(): kehrt Liste um (in-place, gibt None).
- **L.sort(key=None, reverse=False): sortiert in-place (gibt None!).**

- sorted(iterable, key=..., reverse=...): liefert neue Liste!
- reversed(seq): liefert Iterator!

```
L = [3, 1, 4, 1, 5]
L.append(9)          # [3, 1, 4, 1, 5, 9]
L.extend([2, 6])     # [3, 1, 4, 1, 5, 9, 2, 6]
L.insert(1, 99)      # an Index 1 einfügen
last = L.pop()       # 6 (letztes Element)
L.remove(99)         # entfernt 99
L.sort()             # [1, 1, 2, 3, 4, 5, 9]
(in-place!)
neu = sorted(L, reverse=True) # [9, 5, 4, 3, 2, 1, 1]
```

18. SORTIEREN & CUSTOM KEYS

Sortieren nach benutzerdefinierten Kriterien:

```
words = ['Banane', 'Apfel', 'Kiwi']
sorted(words, key=len) # ['Kiwi', 'Apfel', 'Banane']
sorted(words, key=str.lower) # case-insensitive
# Nach Tupel-Element / Attribut sortieren:
data = [('A', 3), ('B', 1), ('C', 2)]
sorted(data, key=lambda x: x[1]) # nach Zahl
# Mehrstufig (z.B. nach Alter, dann Name):
people = [('Mueller', 30), ('Meier', 25), ('Meier', 30)]
sorted(people, key=lambda p: (p[0], -p[1]))
# DSU (Decorate-Sort-Undecorate / Schwartzian):
raw = ['a', 'c', 'b']
[o[2] for o in sorted([(w.lower(), i, w) for i, w in enumerate(raw)])]
```

19. TUPEL & SEQUENZ-ENTPACKEN

- **Unveränderlich (Immutable):** t = (1, 2, 3).
- **Komma-Falle: (42) ist int! Tupel mit 1 Element braucht Komma: (42),!**
- **Unpacking & Star:**

```
a, b = 10, 20          # Vertauschen: a, b = b, a
first, *mid, last = [1, 2, 3, 4, 5]
# first=1, mid=[2, 3, 4], last=5
# NamedTuple (Lesbarkeit von Objekten):
from collections import namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(10, 20); print(p.x, p.y) # 10 20
```

20. FLACHE VS. TIEFE KOPIEN

- **Flache Kopie (Shallow):** c = L.copy() oder c = L[:] (kopiert nur äußere Liste; verschachtelte Objekte werden geteilt!)
- **Tiefe Kopie (Deep):** import copy; c = copy.deepcopy(L) (kopiert alle Unterstrukturen rekursiv).
- **2D-Listen Erstellungsfälle:**

```
matrix = [[0]*3]*3      # FALSCH! Zeilen geteilt!
matrix[0][0] = 1        # ändert ALLE Zeilen!
# Richtig mit List Comprehension:
matrix = [[0 for _ in range(3)] for _ in range(3)]
```

21B. CHUNKING & LISTEN-ROTATION

Nützliche Listen-Algorithmen ohne Fremdmodule:

```
# Liste in Chunks der Größe n teilen:
L = list(range(10)); n = 3
chunks = [L[i:i+n] for i in range(0, len(L), n)]
# [[0,1,2], [3,4,5], [6,7,8], [9]]
# Liste um k Stellen rotieren (zyklisch):
k = 2; rotated = L[k:] + L[:k]
```

22. DICTIONARIES IM DETAIL (METHODEN)

Schlüssel-Wert-Paare. Keys müssen hashbar (immutable) sein.

- **Erstellung:** {}, dict(a=1, b=2), dict(zip(keys, vals)), dict.fromkeys(keys, 0).
- **Falle dict.fromkeys mit Mutables:** dict.fromkeys('ab', []) teilt dieselbe Liste! Stattdessen: {k: [] for k in 'ab'}.
- **Zugriff:** d[k] (KeyError wenn fehlt).
- d.get(k, default=None): sicheres Lesen ohne Fehler!
- d.setdefault(k, default): liest; setzt d[k]=default falls fehlend.
- **Löschen:** del d[k], d.pop('k', def), d.popitem() (LIFO).
- **Update:** d.update(d2) (überschreibt bestehende, fügt neue hinzu).
- **Views:** d.keys(), d.values(), d.items().

```
d = {'Anna': 30, 'Bob': 25}
d.get('Cara', 0)        # 0 (kein KeyError!)
d.setdefault('Dan', 20) # liefert 20 & trägt ein
alter = d.pop('Bob')     # 25 (entfernt 'Bob')
for k, v in d.items(): print(f'{k}: {v}')
```

23. MODERNE DICT-FEATURES & MERGE

- **Dict-Merge Operator** (| und |= ab Python 3.9):

```
d1 = {'a': 1, 'b': 2}
d2 = {'b': 99, 'c': 4}
d3 = d1 | d2          # {'a': 1, 'b': 99, 'c': 4}
d1 |= d2              # modifiziert d1 in-place
# Ältere Versionen / Dict Unpacking:
d_merged = {**d1, **d2, 'extra': 10}
# Dict nach Werten sortieren:
sorted_by_val = sorted(d.items(), key=lambda x: x[1])
```

24. MENGEN (SETS & FROZENSETS)

Ungeordnet, keine Duplikate, ultraschnelle in-Prüfung (O(1)).

- s = {1, 2, 3} | s.add(x) | s.discard(x) (kein Fehler!) | s.remove(x) (KeyError).
- frozenset(seq): unveränderlich (kann Dict-Key sein!).

- **Mengen-Operatoren:**

```
A = {1, 2, 3}; B = {3, 4, 5}
A | B      # {1, 2, 3, 4, 5} (Vereinigung / union)
A & B      # {3} (Schnitt / intersection)
A - B      # {1, 2} (Differenz)
A ^ B      # {1, 2, 4, 5} (Symmetrische Differenz)
A <= B     # False (Teilmenge / issubset)
# Duplikate aus Liste entfernen (ohne Reihenfolge):
unique_items = list(set([1, 2, 2, 3, 1])) # [1, 2, 3]
```

25. HÄUFIGKEITSANALYSE OHNE MODULE

Klassisches Zählen von Elementen in reinem Python:

```
words = ['auto', 'bahn', 'auto', 'zug', 'auto']
# Methode 1: get() mit Default:
counts = {}
for w in words: counts[w] = counts.get(w, 0) + 1
# counts = {'auto': 3, 'bahn': 1, 'zug': 1}
# Gruppieren in Listen:
by_len = {}
for w in words: by_len.setdefault(len(w), []).append(w)
# by_len = {4: ['auto', 'bahn', 'auto', 'auto'], 3: ['zug']}
```

26. JSON & VERSCHACHELTE DICTIONARIES

- **Modul json (Standardbibliothek):**

```
import json
data = {'autos': [{'model': 'Tesla', 'kwh': 75}]}
json_str = json.dumps(data, indent=2) # Serialisieren
parsed = json.loads(json_str)         # Deserialisieren
# Verschachtelten Wert sicher auslesen:
val = parsed.get('autos', [{}])[0].get('kwh', 0)
```

26B. DICT-LÖSCHEN & UNHASHABLE FALLEN

Typische Fallstricke bei Kollektionen:

```
# FALSCH: Löschen während for-Iteration wirft
RuntimeError!
# RICHTIG: Über Kopie der Keys iterieren:
d = {'a': 1, 'b': -2, 'c': 3}
for k in list(d.keys()):
    if d[k] < 0: del d[k]
# Unhashable Type: Listen/Sets dürfen NIE Dict-Keys sein!
# Tupel und Frozensets sind hashbar und erlaubt!
```

27. ITERATOREN & GENERATOREN

- **Iterator-Protokoll:** it = iter(obj) | next(it, [default]).
- **Generatoren mit yield:** erzeugen Daten on-the-fly, speichern Ausführungszustand, sparen Speicher:

```
def count_up(n):
    val = 1
    while val <= n:
        yield val; val += 1
gen = count_up(3)
next(gen) # 1 | next(gen) # 2 | next(gen) # 3
# Generator Expression (runde Klammern!):
squares = (x**2 for x in range(1000000)) # 0 Bytes
Speicher!
# Auswertung erst bei Iteration / sum(squares)
```

28. EINGebaUTE ITERATIONSHELPER

- **enumerate(seq, start=0):** liefert (index, element).
- **zip(*iterables):** parallel iterieren (stoppt bei kürzestem!).
- **all() / any():**

```
cars = ['Tesla', 'BMW', 'Audi']
for idx, name in enumerate(cars, start=1):
    print(f'{idx}: {name}') # '1: Tesla', ...
# Parallel über 2 Listen:
xs = [1, 2, 3]; ys = [10, 20, 30]
coords = [(x, y) for x, y in zip(xs, ys)]
# all / any:
all([True, True, False]) # False (alle müssen wahr sein)
any([False, True, False]) # True (mindestens eins wahr)
```

29. ITERTOOLS POWER-WERKZEUGE

Standardmodul für fortgeschrittene Iterationen:

```
import itertools
# zip_longest (füllt kürzere Listen auf):
itertools.zip_longest([1, 2], ['a', 'b', 'c'], fillvalue='-')
# chain (verbindet mehrere Iterables nahtlos):
list(itertools.chain([1, 2], [3, 4])) # [1, 2, 3, 4]
# islice (Slicing für Generatoren/Iteratoren):
list(itertools.islice(range(100), 2, 6)) # [2, 3, 4, 5]
# Permutationen & Kombinationen:
list(itertools.permutations([1, 2], 2)) # [(1,2),(2,1)]
# Invertieren eines Dicts:
list(itertools.combinations([1, 2, 3], 2)) # [(1,2),(1,3),(2,3)]
```

30. COMPREHENSIONS KOMPLETT

Kompakte, performante Objekterstellung:

```
# List Comprehension: [expr for x in seq if cond]
evens = [x for x in range(10) if x % 2 == 0]
# If-Else im Ausdruck (Ternary):
sign = ['pos' if x >= 0 else 'neg' for x in [-1, 2, -3]]
# Verschachtelte Schleifen (Flattening):
matrix = [[1, 2], [3, 4]]
flat = [val for row in matrix for val in row] # [1, 2, 3, 4]
# Dict Comprehension: {k: v for ...}
sq_dict = {x: x**2 for x in range(5)}
# Invertieren eines Dicts:
inv = {v: k for k, v in {'a': 1, 'b': 2}.items()}
# Set Comprehension: {expr for ...}
unique_mods = {x % 3 for x in [1, 2, 3, 4, 5, 6]} # {0, 1, 2}
```

31. GENERATOR-PIPELINES (VERKETTUNG)

Verkettete Datenverarbeitung ohne Speicherverbrauch:

```
def read_lines(fn):
    with open(fn) as f:
        for l in f: yield l.strip()
def filter_comments(lines):
    for l in lines: if l and not l.startswith('#'): yield l
# total = sum(float(x) for x in filter_comments(read_lines('d.txt')))
```

31B. ITERTOOLS.GROUPBY & VERKETTUNG

Gruppieren sortierter Sequenzen:

```
import itertools
# WICHTIG: Daten MÜSSEN nach Gruppierschlüssel sortiert sein!
cars = [('BMW', 'i4'), ('BMW', 'iX'), ('Tesla', '3')]
for make, group in itertools.groupby(cars, key=lambda x: x[0]):
    models = [m for _, m in group]
    print(f'{make}: {models}')
```

PYTHON 3 CHEAT-SHEET | BLATT 2 (SEITE 3/4): FUNKTIONEN, OOP & OPERATOREN

ACHTUNG: Veränderliche Klassenattribute teilen Zustand aller Instanzen! Mutable Defaults in Funktionen verboten!

32. FUNKTIONEN & PARAMETERÜBERGABE

• Signatur & Parameterarten:

```
def f(pos_only, /, standard, *args, kw_only,
**kwargs):
    # pos_only: darf nur positional übergeben
    # standard: positional oder mit name=wert
    # *args: zusätzliche Positionsargumente (als
    # Tupel)
    # kw_only: MUSS mit name=wert übergeben werden
    # **kwargs: zusätzliche Keyword-Args (als Dict)
    return standard, args, kwargs
# Argument-Entpacken beim Aufruf:
args_list = [10, 20]; kwargs_dict = {'kw_only': 99}
f(1, *args_list, **kwargs_dict)
```

• Rückgabe: return ohne Wert liefert immer None.

37. KLASSEN & OOP-GRUNDLAGEN

Objektorientierung in Python:

```
class Car:
    wheels = 4 # Klassenattribut (FÜR ALLE
    # GEMEINSAM!)
    def __init__(self, model, battery_size):
        # Instanzattribute (gehören nur DIESER
    # Instanz):
        self.model = model
        self.battery_size = float(battery_size)
        self.level = self.battery_size
    def drive(self, km):
        self.level -= km * 0.2; return km
c = Car('Tesla', 75.0)
c.drive(100) # c.level ist nun 55.0
```

• self: Referenz auf das aktuelle Objekt; muss 1. Parameter jeder Instanzmethode sein!

42. VERERBUNG & SUPER()

Vererbung und Erweiterung von Klassen:

```
class StationCar(Car):
    def __init__(self, model, battery, kw):
        super().__init__(model, battery) #
    # Eltern-Init
    self.charging_power = float(kw)
    def charge_time(self, kwh):
        return kwh / self.charging_power
    def drive(self, km):
        actual = super().drive(km)
        print(f'{self.model} fuhr {actual} km')
        return actual
isinstance(StationCar('B',50,11), Car) # True
issubclass(StationCar, Car) # True
```

33. TÖDLICHE FALLE: MUTABLE DEFAULTS!

Default-Werte werden einmalig bei der Funktionsdefinition ausgewertet! Niemals veränderliche Typen ([, {}) als Default setzen!

```
# FALSCH (teilt Liste über alle Aufrufe):
def append_to(val, L=[]):
    L.append(val); return L
append_to(1) # [1] | append_to(2) # [1, 2] ! FEHLER
!
# RICHTIG (Sentinel None verwenden):
def append_to(val, L=None):
    if L is None: L = []
    L.append(val); return L
# Closure (Funktion behält äußeren Zustand):
def make_multiplier(factor):
    def mul(x): return x * factor
    return mul
double = make_multiplier(2); double(5) # 10
```

38. KAPSELUNG, PROPERTIES & SETTER

• Konventionen: `_var` (geschützt), `__var` (Name Mangling: wird zu `_Class_var`).
• `@property` (Getter & Setter mit Validierung):

```
class Battery:
    def __init__(self, cap): self.capacity = cap
    @property
    def capacity(self): return self._cap
    @capacity.setter
    def capacity(self, val):
        if val < 0: raise ValueError('Kapazität
        negativ!')
        self._cap = float(val)
b = Battery(50.0); b.capacity = 60.0
```

39. METHODENARTEN (@CLASSMETHOD, @STATIC)

Unterschiedliche Methoden-Typen in Klassen:

```
class Coordinate:
    def __init__(self, x, y): self.x, self.y = x, y
    # 1. Instanzmethode (erhält self):
    def dist(self): return (self.x**2 +
    self.y**2)**0.5
    # 2. Klassenmethode (erhält cls - ideal als
    # Factory):
    @classmethod
    def from_string(cls, s):
        x, y = map(float, s.split(','))
        return cls(x, y)
    # 3. Statische Methode (ohne self / cls):
    @staticmethod
    def is_valid(x, y): return x >= 0 and y >= 0
```

43. MAGIC METHODS (DUNDER METHODS)

Überladen von Standard-Operatoren & Protokollen:

```
class Vector:
    def __init__(self, x, y): self.x, self.y = x, y
    # String-Repräsentation (Entwickler / Debug):
    def __repr__(self): return f'Vector({self.x},
    {self.y})'
    # Für print() und str():
    def __str__(self): return
    f'({self.x}|{self.y})'
    # Addition (+): self.__class__ für
    # Vererbungssicherheit!
    def __add__(self, o):
        return self.__class__(self.x + o.x, self.y
    + o.y)
    # Vergleich (<) - zwingend für sort() & min():
    def __lt__(self, o): return
    (self.x**2+self.y**2) < (o.x**2+o.y**2)
    # Gleichheit (==):
    def __eq__(self, o): return (self.x, self.y) ==
    (o.x, o.y)
    # Container: len(v), x in v:
    def __len__(self): return 2
    def __getitem__(self, idx): return [self.x,
    self.y][idx]
```

34. LAMBDA & FIRST-CLASS FUNCTIONS

• Lambda (Anonyme Funktion): `f = lambda x, y: x + y`.
• Funktionen als First-Class Citizens:

```
# map(fn, seq) -> Generator für Transformation:
list(map(lambda x: x**2, [1, 2, 3])) # [1, 4, 9]
# filter(fn, seq) -> Generator für Filterung:
list(filter(lambda x: x > 0, [-2, 0, 3])) # [3]
# Dispatch-Tabelle:
ops = {'+': lambda a,b: a+b, '*': lambda a,b: a*b}
ops['+'](10, 5) # 15
```

35. DEKORATOREN & INTROSPEKTION

Dekoratoren modifizieren Funktionen ohne

Quellcode-Änderung:

```
import functools
def my_decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        res = func(*args, **kwargs)
        return res
    return wrapper
# Introspektion:
callable(len) # True
hasattr(obj, 'x') # True wenn Attribut
existiert
val = getattr(obj, 'x', default_val) # sicheres
Lesen
```

36. FUNKTIONALE HELFER (FUNCTOOLS)

• Standardmodul `functools`:

```
from functools import partial, lru_cache
# partial: Parameter vorkonfigurieren:
base2_int = partial(int, base=2)
base2_int('1010') # 10
# lru_cache: automatisches Memoization/Caching:
@lru_cache(maxsize=128)
def fib(n): return n if n < 2 else fib(n-1) +
fib(n-2)
```

40. DYNAMISCHE ATTRIBUTE & REFLECTION

Zugriff auf interne Attribute und Wörterbücher:

```
p = Coordinate(3.0, 4.0)
vars(p) # {'x': 3.0, 'y': 4.0}
p.__dict__ # identisch zu vars(p)
setattr(p, 'z', 10.0); getattr(p, 'z') # 10.0
# Universelles __repr__ für ALLE Attribute:
def __repr__(self):
    args = (f'{k}={v!r}' for k, v in
    vars(self).items())
    return f'{type(self).__name__}({",
    ".join(args)})'
# Alle Methoden ohne Dunder auflisten:
[m for m in dir(p) if not m.startswith('__')]
```

44. FEHLERBEHANDLUNG (EXCEPTIONS)

• Struktur: `try ... except ... else ... finally`:

```
try:
    val = int(input_str)
    res = 100 / val
except (ValueError, TypeError) as e:
    print(f'Eingabefehler: {e}')
except ZeroDivisionError:
    print('Division durch Null!')
else:
    print(f'Erfolg: {res}') # nur wenn KEIN Fehler!
finally:
    print('Aufräumen...') # läuft IMMER!
# Eigene Exception definieren (erbt von
# Exception!):
class CarError(Exception): pass
if fuel < 0: raise CarError('Tank leer!')
```

36B. REKURSION & REKURSIONSGRENZE

• Rekursionsmuster: Basisfall + rekursiver Aufruf:

```
def fakultaet(n):
    if n <= 1: return 1 # Basisfall!
    return n * fakultaet(n - 1)
# Rekursionslimit (Default 1000 Aufrufe):
import sys; sys.setrecursionlimit(2000)
```

40C. PRIVATE ATTRIBUTE & NAME MANGLING

• `_geschuetzt`: Hinweis 'nur intern nutzen' (Konvention).
• `__privat`: Name Mangling wird zu `_KlassenName__privat`.

```
class Bank:
    def __init__(self, pin): self.__pin = pin
b = Bank(1234)
# b.__pin wirft AttributeError!
b._Bank__pin # 1234 (Zugriff über gemangelten
Namen)
```

45C. EIGENE EXCEPTION-HIERARCHIE

Strukturierte Fehler für Bibliotheken:

```
class ExamError(Exception): pass
class FileFormatError(ExamError): pass
# Aufruf:
try: raise FileFormatError('Spalte fehlt!')
except ExamError as e: print(f'Abgefangen: {e}')
```

PYTHON 3 CHEAT-SHEET | BLATT 2 (SEITE 4/4): 2025 KLAUSUR-BLUEPRINTS & PRAXIS

KLAUSUR-STRATEGIE: Schreib ununterbrochen Code! Teillogik bringt viele Punkte, auch wenn ein Detail nicht kompiliert!

46. DATEI-I/O & KONTEXT-MANAGER

• **Sicheres Öffnen mit with:** schließt Datei garantiert automatisch:

```
# Lesen:
with open('daten.txt', 'r', encoding='utf-8') as f:
    text = f.read() # ganze Datei als String
    # ODER zeilenweise lesen (speicherschonend):
    # for line in f: ...
# Schreiben ('w' überschreibt, 'a' hängt an):
with open('out.txt', 'w', encoding='utf-8') as f:
    f.write('Erste Zeile\n')
    f.writelines(['A\n', 'B\n'])
# Cursor steuern: f.seek(0) (an Anfang springen)
```

47. DATEIPFADE MIT PATHLIB & OS

Plattformunabhängige Pfadbehandlung:

```
from pathlib import Path
p = Path('data') / 'messung.csv' # / Operator verbindet!
p.exists() # True wenn Datei vorhanden
p.is_file() # True
text = p.read_text(encoding='utf-8') # Schnell-Lesen
p.write_text('Inhalt', encoding='utf-8')
# os.path Alternative:
import os
path = os.path.join('data', 'messung.csv')
```

48. CSV-PARSING VON HAND (KLAUSUR-MUSTER)

Klausur-Kernkompetenz: Zeile für Zeile ohne CSV-Modul:

```
def parse_csv(filepath):
    rows = []
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f:
            s = line.strip()
            if not s or s.startswith('#'): continue
            p = [x.strip() for x in s.split(',')]
            if len(p) >= 2:
                rows.append((p[0], float(p[1])))
    return rows
```

49. MODULE, PAKETE & CLI

Modul- und Paketstruktur in Python:

```
# Verzeichnisstruktur eines Pakets 'height':
# height/ -> __init__.py, read.py, calc.py
# In height/__init__.py (Öffentliche API):
from .read import read_csv_file
from .calc import mean, shortest_person, tallest_person
__all__ = ['read_csv_file', 'mean', 'shortest_person']
# Importpfad zur Laufzeit erweitern:
import sys
sys.path.append('.') # aktuelles Verzeichnis
args = sys.argv[1:] # CLI Argumente
```

50. STANDARDBIBLIOTHEK POWER-MODULE

Wichtige Module ohne Installation:

```
# random: Zufallszahlen & Auswahl:
import random
random.randint(1, 6) # Zufalls-Int von 1 bis 6
random.choice(['A', 'B']) # Zufallselement
random.shuffle(L) # mischt Liste in-place!
# collections: Häufigkeiten & Standardwerte:
from collections import defaultdict, Counter
d = defaultdict(list); d['k'].append(1)
counts = Counter(['a', 'b', 'a']).most_common(1)
```

50B. ZEIT & DATUM (MODUL DATETIME)

Zeitstempel parsen & formatieren (Standard-Lib):

```
from datetime import datetime, timedelta
now = datetime.now()
stamp = now.strftime('%Y-%m-%d %H:%M:%S') # Formatieren
dt = datetime.strptime('2026-03-15', '%Y-%m-%d') # Parsen
tomorrow = now + timedelta(days=1) # Rechnen mit Zeit
```

51. KLAUSUR 2025: AUFGABE 1 (SYNTAX & TYPEN)

Aufgabe 1 (100% aus Gedächtnisprotokoll):

```
# 1. Liste L mit Start, Ende, Schritte:
L = list(range(100, 2301, 200))
# [100, 300, 500, ..., 2300]
# 2. Liste umkehren (Zusatzpunkte für mehrere Arten!):
# Art 1 (Slicing): L[::-1]
# Art 2 (reversed): list(reversed(L))
# Art 3 (in-place): L_rev = L.copy(); L_rev.reverse()
# Art 4 (sorted): sorted(L, reverse=True)
# Art 5 (Comp): [L[-i] for i in range(len(L))]
# 3. Dict mit string.ascii_uppercase ab 'D':
import string
d = dict(zip(string.ascii_uppercase[3:], L))
# d = {'D': 100, 'E': 300, ..., 'O': 2300}
# 4. Sets Schnitt/Diff/Vereinigung (Methoden & Operatoren):
s1, s2 = {1, 2, 3, 4}, {3, 4, 5, 6}
schnitt = s1 & s2 # s1.intersection(s2)
diff1 = s1 - s2 # s1.difference(s2)
diff2 = s2 - s1 # s2.difference(s1)
vereinigung = s1 | s2 # s1.union(s2)
# 5. 1-Zeilen Key-Zugriff & Fehlerbehandlung:
# a) KeyError -> ValueError werfen:
val = d[k] if k in d else (_ for _ in ().throw(ValueError))
# b) Key fehlt -> None zurückgeben:
val = d.get(k)
# c) Key fehlt -> beliebiges Objekt zurückgeben:
val = d.get(k, 'DEFAULT_OBJ')
# 6. Formatierung (a=100, b=2.57878):
a, b = 100, 2.57878
f'{a} {b}' # '100 2.57878'
f'{a} {b}' # '100 2.57878 100'
f'{b:>5.2f}' # ' 2.58' (Breite 5!)
f'{a:07d}' # '0000100' (7-stellig)
f'{a*2}' # '200'
```

52. KLAUSUR 2025: AUFGABE 2 (PERSONEN.CSV)

Aufgabe 2 (CSV, Spaltentausch & Min/Max):

```
# CSV einlesen mit automatischer Spaltentausch-Erkennung:
def read_csv_file(filename):
    names, heights = [], []
    with open(filename, 'r', encoding='utf-8') as f:
        for line in f:
            p = [x.strip() for x in line.strip().split(',')]
            if not p or not p[0] or p[0].startswith('#'): continue
            try:
                # Test ob Spalte 0 Zahl ist (Größe, Name):
                h = float(p[0]); n = p[1]
            except ValueError:
                # Spalte 0 ist Name (Name, Größe):
                try: n = p[0]; h = float(p[1])
            except ValueError: continue # Header
            names.append(n); heights.append(h)
    return names, heights # Bonus Dict:
dict(zip(names, heights))

def mean(h): return sum(h) / len(h)
def shortest_person(n, h):
    i = h.index(min(h)); return n[i], h[i]
def tallest_person(n, h):
    i = h.index(max(h)); return n[i], h[i]
# Bonus: min oder max als First-Class-Funktion übergeben:
def min_max_person(n, h, func):
    target = func(h); i = h.index(target); return n[i], target
# Bonus: Mehrere Personen mit gleicher Min/Max-Größe (Ties):
def min_max_persons(n, h, func):
    target = func(h)
    return [name for name, val in zip(n, h) if val == target]
```

52B. BLUEPRINT: PROJEKT 'COORDS'

Komplettes Koordinaten-Modul mit Vektorrechnung:

```
import math
class Coord:
    def __init__(self, x, y): self.x, self.y = float(x), float(y)
    def distance(self): return math.hypot(self.x, self.y)
    def __add__(self, o): return self.__class__(self.x + o.x, self.y + o.y)
    def __lt__(self, o): return self.distance() < o.distance()
    def __repr__(self): return f'Coord({self.x}, {self.y})'
# Summen über Listen & Dicts:
def sum_lists(xs, ys): return sum(xs), sum(ys)
def sum_dict(d): return sum(d['X']), sum(d['Y'])
```

53. KLAUSUR 2025: AUFGABE 3 (OOP)

Aufgabe 3 (Klassen, Vererbung & dynamischer Repr):

```
class Person:
    def __init__(self, name, height):
        self.name = str(name)
        self.height = float(height)
    def grow(self, amount):
        self.height += amount
    # Dynamische Ausgabe ohne neues __repr__ in Subklasse:
    def __repr__(self):
        cls = self.__class__.__name__
        return f'{cls}({self.name}, {self.height})'

class BesserePerson(Person):
    def grow(self, amount):
        if not (0 < amount < 10):
            raise ValueError('Wachstum 0 < x < 10!')
        super().grow(amount)

# Dictionary von Personen-Instanzen aus A2:
persons = {n: Person(n, h) for n, h in zip(names, heights)}
```

54. KLAUSUR 2025: AUFGABE 4 (PAKET)

Aufgabe 4 (Paketstruktur 'height'):

```
# Verzeichnisbaum:
# height/ -> __init__.py, read.py, calc.py
# In height/__init__.py:
from .read import read_csv_file
from .calc import mean, shortest_person, tallest_person
__all__ = ['read_csv_file', 'mean', 'shortest_person', 'tallest_person']
# Geforderte Importe:
from height.read import read_csv_file
from height.calc import mean
# Importe mit Alias:
import height.read as hr
import height.calc as hc
```

54B. BLUEPRINT: PROJEKT 'ECARS'

Komplettes E-Auto Klassenmodell mit Vererbung:

```
class Car:
    def __init__(self, model, bat, cons):
        self.model, self.battery_size, self.consumption = model, float(bat), float(cons)
        self.battery_level = self.battery_size
    def drive(self, km):
        max_km = (self.battery_level / self.consumption) * 100
        actual = min(km, max_km)
        self.battery_level -= (actual * self.consumption) / 100
        return actual
    def charge(self, kwh):
        self.battery_level = min(self.battery_size, self.battery_level + kwh)
    def __lt__(self, o): return self.battery_size < o.battery_size
    def __repr__(self): return f'Car({self.model!r}, {self.battery_size})'
```

55. TOP 10 PRÜFUNGS-FALLEN & NOTFALL-JOKER

- In-Place Methoden:** L.sort() gibt None zurück! Nie L = L.sort()!
- Mutable Defaults:** def f(L=[]): teilt Zustand! Immer L=None!
- dict.fromkeys:** dict.fromkeys('ab', []) teilt dieselbe Liste!
- Float-Vergleiche:** Nie ==, immer math.isclose()!
- Division:** / liefert float; // liefert ganzzahlig.
- is vs ==:** is prüft Identität (id), == prüft Wert.
- Klassenattribute:** Listen im Klassenkörper teilen Zustand aller Instanzen!
- 2D-Listen:** [[0]*3]*3 wiederholt selbe Zeile!
- CSV-Header:** Immer mit startswith('#') und strip() filtern.
- NOTFALL-JOKER:** Bei Teilaufgaben-Fehlern Zwischenergebnisse in data/*.txt per Copy-Paste nutzen!

56. CHECKLISTE: WINDOWS & H-LAUFWERK

- **Kernel-Reset vor Abgabe:** Kernel → Restart Kernel and Run All Cells im JupyterLab ausführen und Output prüfen!
- **Tastatur (Win):** l: AltGr + 6 | |: AltGr + < | {}: AltGr + 7 / 0 | []: AltGr + 8 / 9
- **Arbeitsverzeichnis prüfen:** %pwd in Notebook ausführen.
- **Pfadtrenner:** unter Windows immer / oder r'...' verwenden.
- **DATENSICHERUNG NACH H:** Am Ende alle *.ipynb und *.py zwingend auf das persönliche Netzlaufwerk H: kopieren!
- **KEINE LINKS:** Echte Datei-Kopien nach H: legen, keine *.url oder *.lnk Verknüpfungen!
- **Prüfen:** Im Windows Explorer prüfen, ob Dateien auf H: liegen und > 0 Bytes groß sind!